November 2026 Development Task Plan

A four-feature development cycle for a 2D engine that supports both game development and real-time visualization / simulation.

Direction	Game engine + real-time visualization / simulation engine
Scope	Four major features; build on the existing Rust + WGPU architecture
Delivery approach	One focused feature per week, each developed on a feature branch and validated before merging

Monthly milestones

WEEK 1 | NOV 1–7 — Asset & Resource Manager

Goal: Centralize asset loading and reuse GPU resources instead of repeatedly loading the same files.

Task checklist

- ■ Inspect current texture, sprite, and font ownership/lifetime paths.
- ■ Create an **assets** module with an **AssetManager**.
- ■ Normalize asset paths so equivalent paths resolve to the same cache key.
- ■ Cache GPU textures and return cheap, cloneable texture handles.
- ■ Return actionable errors for missing or invalid assets; avoid panics for normal load failures.
- ■ Load the particle demo texture through the manager and preserve the existing Sprite workflow.
- ■ Add tests for path normalization and cache reuse; document manager ownership.

Definition of done: Repeated requests for the same normalized texture path reuse one GPU resource; missing assets produce clear errors; the demo still renders.

WEEK 2 | NOV 8–14 — Scene Management

Goal: Introduce a simple scene abstraction so the application can organize and switch between distinct worlds or demo states.

Task checklist

- ■ Define a minimal Scene interface / lifecycle (initialize, update, render or equivalent responsibilities).
- ■ Move the current particle visualization setup into a dedicated scene.
- ■ Add a scene manager that owns the active scene and handles transitions.
- ■ Keep window, renderer, and device ownership at the application/engine level.
- ■ Define how scene-owned entities and resources are created and released.
- ■ Add a second lightweight test scene or transition test to prove switching works.

Definition of done: The application can run the particle scene through the new scene abstraction and switch to another scene without recreating the WGPU device.

WEEK 3 | NOV 15–21 — Sprite Sheets & Animation

Goal: Support frame-based 2D animation using texture atlases.

Task checklist

- ■ Define a sprite-sheet representation (texture, frame dimensions/rectangles, frame count).
- ■ Add UV-coordinate selection for an individual frame.
- ■ Implement animation clips with frame duration and playback state.
- ■ Support looping and one-shot playback, with a clear completion state.
- ■ Integrate animated sprites with the existing Sprite/rendering path.
- ■ Create a small animation test scene and verify frame timing at variable frame rates.

Definition of done: A sprite sheet can be loaded and played as a looping or one-shot animation with stable delta-time-based frame progression.

WEEK 4 | NOV 22–30 — Basic 2D Collision Detection

Goal: Add foundational collision queries suitable for simple games without introducing a full physics engine.

Task checklist

- ■ Define basic 2D collider data for axis-aligned bounding boxes (AABB) and circles.
- ■ Implement AABB-vs-AABB overlap tests.
- ■ Implement circle-vs-circle overlap tests.
- ■ Implement AABB-vs-circle overlap tests.
- ■ Expose collision results/events to game or scene logic.
- ■ Add unit tests for overlap, separation, containment, and edge-touching cases.
- ■ Document coordinate-space assumptions and collider ownership.

Definition of done: The supported shape pairs return correct, tested overlap results and can be queried by scene/game logic.

Engineering rules & delivery checklist

These constraints keep November focused on reusable engine foundations rather than a large rewrite.

Architecture	Extend the current modular Rust/WGPU codebase incrementally. Do not rewrite the renderer or introduce a full ECS/physics architecture as part of this month.
Ownership	Keep WGPU device/queue ownership in the renderer/application layer. Resource and scene abstractions should use explicit ownership and lifetimes.
Compatibility	Preserve the October Particle Flow Field demo while adding general-purpose engine capabilities.
Scope control	Defer asynchronous asset loading, hot reload, asset bundles, advanced scene graphs, skeletal animation, full physics simulation, and 2D lighting.
Branch workflow	Create each feature branch from the latest stable master. Validate it, then merge through a pull request after review.

Per-feature release gate

- ■ Code is formatted with **cargo fmt**.
- ■ Project passes **cargo check** and relevant **cargo test** tests.
- ■ Existing particle visualization still starts and renders correctly.
- ■ New functionality has focused tests and concise README/API documentation.
- ■ No unrelated refactors or unplanned features are included in the branch.
- ■ Feature branch is merged only after local validation and review.

Deferred to a later milestone: tilemaps, layer/depth ordering, sprite batching, 2D lighting, UI framework, audio, generalized particle emitters, full physics, and 3D rendering.